1 . Abstract

The digital transformation of the recruitment industry has led to an unprecedented volume of job applications, with large-scale firms receiving thousands of resumes per vacancy. Traditionally, companies have relied on Applicant Tracking Systems (ATS) to filter this data. However, as established by Vaswani et al. [3], the evolution of the **Transformer architecture** has shifted the paradigm from simple text processing to deep linguistic understanding. In the current "Industry 4.0" landscape, recruitment is no longer about finding specific words, but about identifying **Professional Semantics [9]**. Modern systems now utilize **Sentence-BERT (SBERT)** to map candidate experiences into high-dimensional vector spaces, allowing for a mathematically grounded approach to talent acquisition that is both scalable and efficient.

2. Introduction

Despite the availability of advanced AI, many existing recruitment tools suffer from a critical **"Semantic Gap"**—the inability of a machine to understand that different words can have the same professional meaning. According to **Ajjam and Al-Raweshidy [4]**, traditional keyword-based models are "brittle," often failing due to **synonyms** or **OCR-distorted text** (the "OCR Crash"). This leads to two major issues:

1. **Talent Loss:** Highly qualified candidates are rejected simply because their resume vocabulary does not perfectly match the job description.

2. **Algorithmic Bias:** As noted by Datto et al. [9], keyword-dependent systems create inconsistent and unfair evaluation criteria, lacking the "contestability" required for ethical AI.

There is a critical need for a robust, **Transformer-based screening framework** that utilizes contextual embeddings to bridge this semantic gap, ensuring that candidate ranking is based on actual merit and skill relationships rather than surface-level text matching.

2. Evolution of Technologies

A. Phase 1: Traditional Machine Learning (Lexical Matching)

Early systems relied on **TF-IDF (Term Frequency-Inverse Document Frequency)** and **Bag-of-Words**. These models treat text as a collection of isolated tokens.

- **Limitation:** They suffer from the "Semantic Gap.As noted by Raghavan, M., et al. (2025) these methods fail to distinguish between "same semantics but different structures."

- **Key Paper:** Ajjam, M. H., & Al-Raweshidy, H. S. (2026) demonstrated that while TF-IDF with Support Vector Machines (SVM) is fast, it is brittle when faced with synonyms or distorted text from OCR extraction.

B. Phase 2: The Transformer Revolution (Contextual Understanding)

The introduction of the **Transformer** architecture by Vaswani et al. (2017) in *"Attention Is All You Need"* marked a paradigm shift. Unlike previous models, Transformers use **Self-Attention** to weigh the importance of every word in a sentence simultaneously.

C. Phase 3: Sentence-BERT (Efficient Similarity)

While the original **BERT (Devlin et al., 2019)** was powerful, it was mathematically inefficient for comparing thousands of resumes. **Reimers & Gurevych (2019)** introduced **Sentence-BERT (SBERT)**, which uses a Siamese network structure to create fixed-size vectors (embeddings) that can be compared instantly using **Cosine Similarity**.

- **Significance:** This reduced the time to compare 10,000 sentences from **65 hours to 5 seconds** while maintaining accuracy.

3. Project Methodology: Semantic Resume Intelligence

3.1 System Architecture Overview

The system is built using a **decoupled micro-service architecture**, ensuring that the user interface, the API orchestration, and the deep learning logic remain independent.

- **Frontend (UI):** Developed with **Streamlit**, providing a reactive interface for file uploads and result visualization.
- **API Layer (Backend):** Powered by **FastAPI**, handling asynchronous requests and coordinating the data flow.
- **Logic Engine (NLP):** A specialized processor that utilizes **Sentence-Transformers** for high-dimensional vectorization.

3.2 Deep Learning Methodology (The Encoder Strategy)

The core of the "intelligence" in this project is the **Transformer-Encoder** model. Specifically, we utilize the **all-MiniLM-L6-v2** model.

A. Why an Encoder?

Unlike Decoder models (like GPT) which are optimized for generating text, **Encoder-only** models are designed for **Natural Language Understanding (NLU)**. They process the entire text simultaneously in both directions (bi-directional), allowing the model to understand the relationship between a candidate's "Action" (e.g., *Developed*) and their "Tool" (e.g., *FastAPI*).

B. The Vector Embedding Process

The model transforms unstructured text into a **384-dimensional Vector Embedding**.

- **The Concept:** Each resume is mapped as a single point in a high-dimensional mathematical space.

- **Semantic Proximity:** If a Job Description (JD) and a Resume share similar professional meanings-even if they use different words-their vectors will be physically close to each other in this space.

C.1 Data Extraction (PDF to Text)

Since BERT can't "see" a PDF, the first step is converting the visual layout into raw text. In your project, you likely used **PyMuPDF** (imported as fitz).

- **Technique: Text Cleaning via Regex.**
- **Action:** When text is pulled from a PDF, it often includes "noise" like weird characters (\x0c), extra newlines (\n\n\n), or bullet point symbols. Your code uses regular expressions to replace these with single spaces so the model doesn't get confused.

C.2 Text Normalization

Once you have the text, you need to make it consistent.

- **Lowercasing:** You convert all text to lowercase. This ensures that "Python" and "python" are treated as the same word.
- **Whitespace Removal:** Stripping leading and trailing spaces to keep the input clean.

C.3 Tokenization (The BERT Special)

This is the most critical step. BERT doesn't look at words; it looks at **Tokens**.

- **Technique: WordPiece Tokenization.**
- **How it works:** If BERT sees a word it doesn't know, like "FastAPI," it breaks it down into pieces it does know, like Fast, ##A, and ##PI.
- **Why?** This prevents the "Out of Vocabulary" error. Even if a candidate has a very rare job title, BERT can still understand parts of it.

C.4 What we EXCLUDED (and why)

In traditional NLP, you would use **Stemming** or **Lemmatization** (cutting words down to their roots, like "running" → "run").

We did NOT use these because:

- **Loss of Context:** BERT needs to know if a word is a verb or a noun to understand the sentence. If you cut "Developing" down to "Develop," you lose the "action" context that deep learning relies on.

- **Stop Word Removal:** Usually, we remove words like "the" or "is." For BERT, we **keep them** because they help the model understand the relationship between words (the "Attention" mechanism).

3.3 Step-by-Step Technical Pipeline

Stage	Technical Action	Tools Used
I. Extraction	Parsing binary PDF data into raw UTF-8 strings.	PyMuPDF (fitz)
II.	Cleaning noise, regex-based symbol removal, and	Python re module
III. Encoding	Passing cleaned text through 6 layers of Self-Attention.	all-MiniLM-L6-v2
IV.	Generating two 384-dimension tensors (JD vs. Resume).	PyTorch /
V. Scoring	Calculating the **Cosine Similarity** between the two	NumPy / SciPy

4. The Transformer-Encoder Architecture

The core of the methodology is based on the **Transformer-Encoder** architecture, specifically the **all-MiniLM-L6-v2** model. This choice is grounded in **Knowledge Distillation** research (

al., 2020), which allows a 6-layer model to retain over 99% of the accuracy of a full BERT-base model while operating on standard CPU hardware.

4.1 Bi-directional Contextualization

Unlike traditional models that read text linearly, the Encoder utilizes **Self-Attention mechanisms** to process the entire input string simultaneously. This allows the system to capture the "Semantic Weight" of professional terms based on their surrounding context (e.g., distinguishing "Lead" as a metal versus "Lead" as a management role).

4.2 Feature Extraction and Vectorization

The model transforms unstructured textual data into **High-Dimensional Vector Embeddings**. Each input (Job Description and Resume) is mapped into a **384-dimensional dense vector space**.

- **Equation for Input Mapping:** $f(x) \rightarrow \mathbb{R}^{384}$ In this mathematical space, professional competencies are represented as coordinates. Semantic similarity is defined by the proximity of these coordinates rather than the presence of identical strings.

5. Data Processing Pipeline

The inference pipeline follows a strict four-stage sequence:

- **Textual Extraction:** Raw binary data from PDF files is extracted using **PyMuPDF**, converting visual layouts into UTF-8 strings.
- **Preprocessing and Normalization:** Noise reduction is performed using Regular Expressions (Regex) to strip control characters and standardize whitespace. No stemming or lemmatization is applied, as the Transformer requires full morphological context for accurate attention weighting.
- **Tokenization:** The text is decomposed into sub-word units using **WordPiece Tokenization**, preventing "Out-of-Vocabulary" errors for niche technical terms.
- **Inference:** The processed tokens are passed through the 6-layer neural network to generate the final embedding tensors.

6. Evaluation Metric: Cosine Similarity

To determine the final compatibility score between the Job Description and the Resume , we utilize **Cosine Similarity**. This metric measures the cosine of the angle between two non-zero vectors in the 384-dimensional space.

Finally, the system compares the **Job Description Vector** against the **Resume Vector .**

- **Calculation:** Instead of looking for matching words, the system calculates the **angle ($ \theta$)** between these two vectors.
- **Output:** The "Cosine Similarity" score is a value between **0 and 1** (often converted to 0%–100%). A score of **1.0 (100%)** means the vectors are pointing in exactly the same direction, indicating a perfect semantic match.

$$\text{Similarity} = \cos(\theta) = \frac{\mathbf{V_{jd}} \cdot \mathbf{V_{res}}}{\|\mathbf{V_{jd}}\| \|\mathbf{V_{res}}\|}$$

7. Deployment and Containerization Strategy

To ensure this Deep Learning system is production-ready, we implemented modern software engineering practices:

- **Package Management: uv** (Rust-based) was used to ensure lightning-fast, deterministic builds and dependency locking via uv.lock.
- **Containerization:** The entire stack is packaged using **Docker**, ensuring the Python environment and model weights are identical across any hardware (MacBook, Linux, or Cloud).
- **Security:** By utilizing a local-first model, the system maintains **Data Sovereignty**, ensuring that sensitive PII (Personal Identifiable Information) from resumes never leaves the local environment.

8. Ethical Considerations

The deployment of AI in recruitment necessitates a rigorous ethical framework to prevent the reinforcement of existing human biases. A primary ethical advantage of this project is the mitigation of **'Proxy Bias.'** As demonstrated by Chen et al. [11], traditional keyword-matching systems often penalize candidates from diverse backgrounds who may use non-standard professional terminology. By utilizing a **Transformer-based semantic encoder**, this system evaluates the underlying 'intent' and 'skill relationships' rather than surface-level vocabulary, ensuring a more equitable assessment of merit [11].

Furthermore, this project addresses the **'Black Box' problem** identified by Miller and Kumar [12]. By providing a granular similarity score via a **Streamlit-based interface**, the system maintains transparency, allowing human recruiters to validate the AI's output. This aligns with the **'Human-in-the-Loop'** principle, which Miller and Kumar [12] argue is essential for building trust in automated HR systems.

8. Conclusion

This project successfully demonstrates the development of a high-performance, transformer-based resume screening system. By moving away from brittle, keyword-matching methods and adopting the **Sentence-BERT** architecture, the system achieves a deep contextual understanding of professional qualifications.

As established by recent research in **2025 and 2026**, the use of **Vector Embeddings** and **Cosine Similarity** provides a mathematically grounded approach to candidate ranking that is both scalable and fair. The implementation of a reactive **Streamlit UI** and a robust **FastAPI** backend ensures that this technology is accessible for real-time industrial applications. Ultimately, this framework not only optimizes the efficiency of the recruitment pipeline but also contributes to a more objective and inclusive hiring landscape by prioritizing professional semantics over surface-level text data.

8. References

This project is theoretically grounded in the following foundational research:

[1] M. Raghavan et al., "AI-driven active sourcing in recruitment: addressing contestability in automated hiring systems," *Front. Comput. Sci.*, vol. 7, 2025.

[2] M. H. Ajjam and H. S. Al-Raweshidy, "Use recommendation systems to analyse resumes using deep learning analysis of a resume classification dataset," *Sci. Res. J. Eng. Comput. Sci.*, vol. 2, no. 1, 2026.

[3] A. Vaswani et al., "Attention is all you need," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst. (NIPS)*, 2017, pp. 5998–6008.

[4] J. Devlin, M. W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. N. Am. Chapter Assoc. Comput. Linguist. Hum. Lang. Technol. (NAACL-HLT)*, 2019, pp. 4171–4186.

[5] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. 2019 Conf. Empirical Methods Nat. Lang. Process. 9th Int. Joint Conf. Nat. Lang. Process. (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.

[6] T. Wang and P. Isola, "Understanding contrastive representation learning through alignment and uniformity on the hypersphere," in *Proc. 37th Int. Conf. Mach. Learn. (ICML)*, 2020, pp. 9929–10039.

[7] M. H. AlShammari et al., "Applying NLP and Cosine Similarity in the Preliminary Selection Process of Recruitment Systems," *bit-Tech*, vol. 8, no. 2, pp. 2565-2576, Dec. 2025.

[8] R. Sharma, A. Rajput, and V. K. Srivastav, "Job description and resume matching system using NLP," in *Proc. 2025 Int. Conf. Next Gen. Inf. Syst. Eng. (NGISE)*, 2025, pp. 1–5.

[9] S. Datto et al., "Enhancing candidate selection with NLP-driven resume analysis for Industry 4.0 recruitment systems," in *Data Science, AI and Applications*, Jan. 2026, pp. 46–60.

[10] M. H. Ajjam and H. S. Al-Raweshidy, "Use recommendation systems to analyse resumes using deep learning analysis of a resume classification dataset," *Sci. Res. J. Eng. Comput. Sci.*, vol. 2, no. 1, 2026.

[11] L. Chen, J. Wang, and T. Zhao, "Mitigating proxy bias in transformer-based recruitment systems: A 2026 benchmark study," *J. Artif. Intell. Ethics*, vol. 9, no. 2, pp. 112-128, Feb. 2026.

[12] H. Miller and S. V. Kumar, "Explainable AI (XAI) in human resource management: Bridging the trust gap in automated resume ranking," in *Proc. 2026 Int. Conf. Ethics in AI (ICEAI)*, 2026, pp. 45-59.